

TECNOLÓGICO NACIONAL DE MÉXICO

Gestión de Proyectos

DISEÑO DE ENDPOINTS RESTFUL

Equipo 3A

Bolaños García Omar Gadiel

Diego Emilio Ortiz Vilchez

Perez Manriques Jorge Luis

Chavez Vega Emilio Sebastian

ING. Sistemas Computacionales | Celaya

Proyecto: Monitor de Nivel de Agua en Tinaco

KR 3.3: Diseño de Endpoints RESTful

Entregable: Documentación técnica (API Swagger-like Format)

Sprint 3 | Fecha: 13 may 2026 | **Estado:** In Review

1. Objetivo

Modelar y documentar los puntos de entrada (endpoints) de la API REST que gestionará la comunicación entre el hardware (ESP32), la base de datos (Firebase) y la aplicación móvil. Esta especificación define las rutas, métodos permitidos, esquemas de entrada/salida y códigos de estado HTTP para garantizar una integración robusta y estandarizada.

2. Arquitectura de Endpoints

Se ha optado por una estructura basada en recursos bajo la versión v1 de la API. Dado el uso de Firebase Realtime Database, los endpoints siguen una estructura de nodos jerárquicos. La seguridad se gestiona mediante tokens de autenticación de base de datos (Auth Tokens) en los parámetros de consulta o headers.

3. Especificación de Rutas (Endpoints)

Método	Ruta (Resource)	Descripción	Uso Principal
PUT	/tinaco/lectura.json	Actualiza el estado actual del sensor (sobrescritura).	Hardware (ESP32)
POST	/tinaco/historial.json	Añade una nueva lectura a la lista histórica.	Cloud Functions
GET	/tinaco/lectura.json	Obtiene la última medición disponible en tiempo real.	App Móvil / Web
GET	/tinaco/config.json	Recupera parámetros de calibración (altura, límites).	ESP32 / App

4. Catálogo de Códigos de Respuesta HTTP

Para asegurar el manejo de errores en el microcontrolador y la aplicación móvil, se definen los siguientes códigos de estado:

Código	Significado	Contexto de Aplicación
200 OK	Éxito	Solicitud procesada correctamente (Lectura de datos exitosa).
201 Created	Creado	Nuevo registro en historial generado exitosamente.
400 Bad Request	Error de Cliente	JSON mal formado, falta de campos obligatorios o tipos de datos inválidos.
401 Unauthorized	Sin Auth	Token de acceso inválido o expirado.
404 Not Found	No Encontrado	El recurso solicitado (nodo del tinaco) no existe.
500 Server Error	Error Interno	Fallo en los servicios de Firebase o latencia excesiva.

5. Detalle Técnico: Actualización de Lectura (Hardware)

Request

- **URL:** https://<id>.firebaseio.com/tinaco/lectura.json?auth=<TOKEN>
- **Método:** PUT
- **Cuerpo (Body):** Ver esquema optimizado en KR 3.2.

```
// Ejemplo de Request Body
{
  "d": 25.5,
  "n": 34.5,
  "p": 57.5,
  "a": 0,
  "t": "2026-05-13T09:15:00-06:00",
  "s": {".sv": "timestamp"}
}
```

Response

- **Status:** 200 OK
- **Body:** Retorna el objeto JSON enviado como confirmación.

```
// Ejemplo de Response Body
{
  "d": 25.5,
  "n": 34.5,
  "p": 57.5,
  "a": 0,
  "t": "2026-05-13T09:15:00-06:00",
  "s": 1715613300000
}
```

6. Conclusiones y Reglas de Negocio

- **Idempotencia:** El uso de PUT en el endpoint de lectura asegura que si el ESP32 reintenta una petición fallida por red inestable, no se dupliquen datos innecesarios.
- **Manejo de Errores:** Si el ESP32 recibe un código 401, debe intentar re-autenticarse. Si recibe un 400, debe descartar la lectura y revisar la serialización.
- **Optimización:** Se evita el uso de PATCH para minimizar el overhead de procesamiento en el hardware, prefiriendo la sustitución completa del objeto pequeño definido en el KR 3.2.